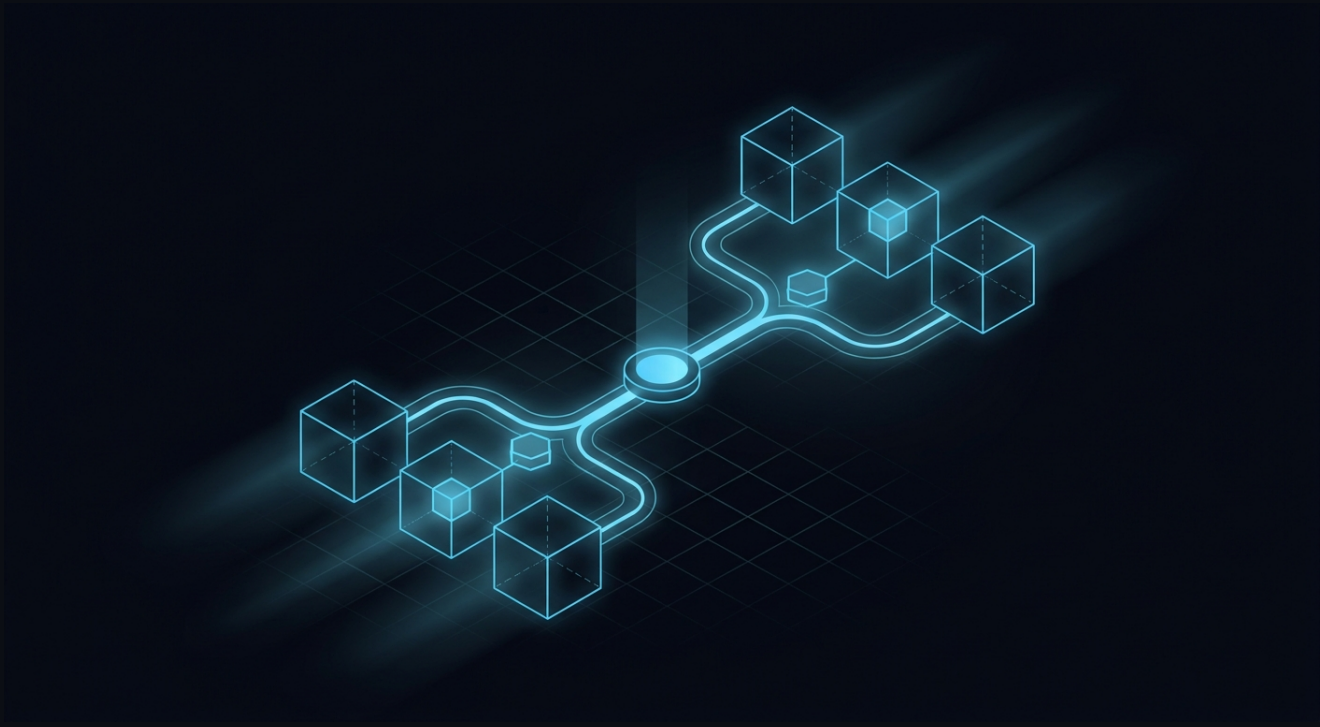
RELEASE PAPER · VERSION 4.0 · APPROVED-FOR-BUILD

# Point-and-Click Dual-Renderer Engine

System Architecture, Core Runtime, Toolchain, Build & Governance Blueprint



A reusable, content-neutral foundation for building any point-and-click adventure — dual-renderer (Babylon + Three) behind one neutral port, shipping to browser and desktop, playable fully offline.

| | |
|---|---|
| Edition | Professional / Commercial-Grade |
| Supersedes | Version 3.0 |
| Format | Designed dark-mode release paper |
| Audience | Engineering, QA, Security, Product, DevOps |

# Contents

# 1. How to Read This Paper

This document is a **content-neutral engineering blueprint**: it defines a reusable engine, editor, content schemas, native shell, persistence, security and compliance posture, build pipeline, governance model, and a complete default webapp game scaffold. It deliberately defines **no** story, characters, art, rooms, or game-specific rules — a real project forks it and drops in content.

It doubles as a **tutorial**. Each major system leads with *why* before *how*, so you can read top-to-bottom to learn the architecture, or jump via the Contents to a specific subsystem.

> ◆ **The central bet**
>
> One platform-neutral TypeScript core. One canonical content model. One RendererPort with two providers (Babylon default, Three specialist). Tauri 2 + Rust for the privileged shell. HTML/CSS for all accessibility-critical UI. Pay for the second engine only when a scene earns it.

**Reading paths.**

- **Architects / leads** — §2 Executive Summary, §3 Audit, §4 Architecture, §12 Roadmap.
- **Engineers** — §4 Architecture, §5 Engineering Quality, §8 Delivery.
- **Security / Privacy** — §6 Security & Threat Model, §7 Privacy & Compliance.
- **QA / DevOps** — §5 Determinism & Observability, §8 CI/CD, §12 Phases & gates.

# 2. Executive Summary

This is a foundation for building any point-and-click adventure — 2D, 2.5D, prerendered, sprite, or full 3D — that ships to the browser and to Windows/macOS/Linux desktop, plays fully offline by default, and can later gain optional online services without an engine rewrite.

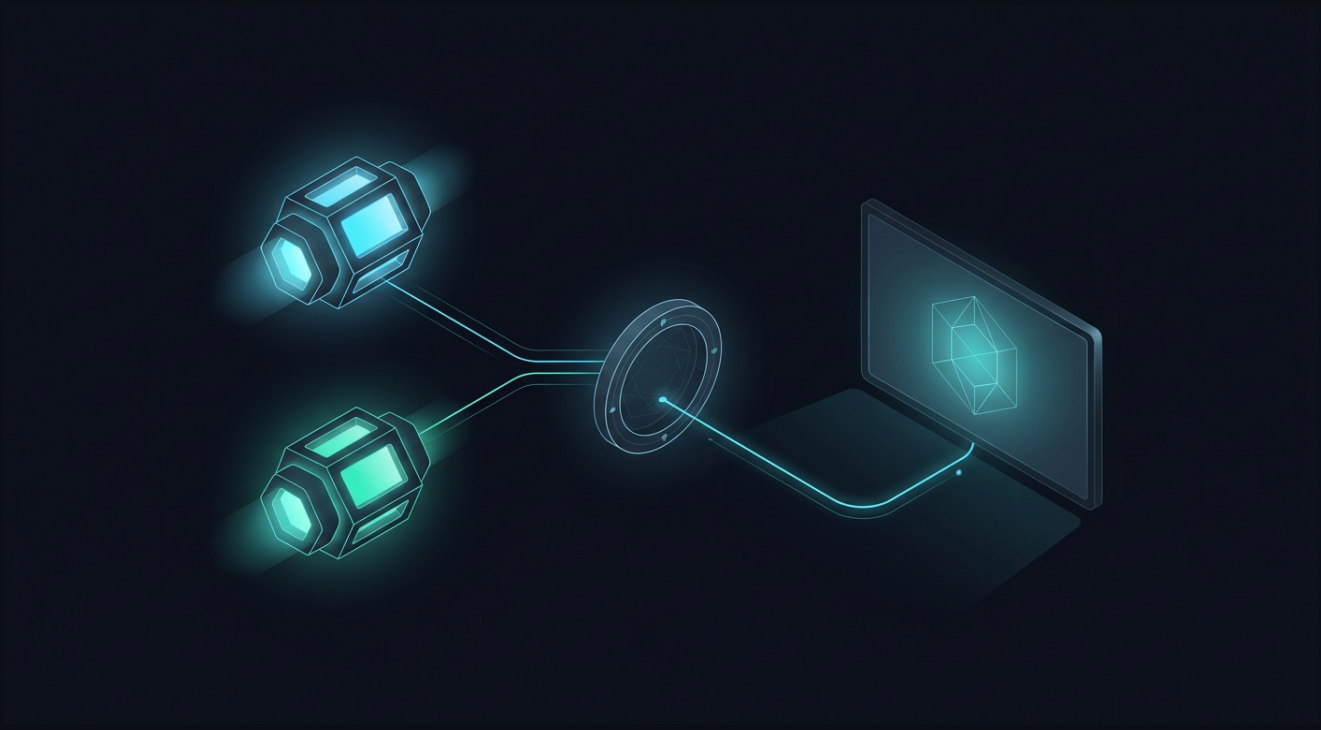


*Figure 2.1 — Two engines, one neutral port, one unified viewport. The core never imports an engine directly.*

**Why v4 exists.** v3 was a strong architecture but read as an engineering spec, not a fundable, auditable, legally shippable program plan. v4 keeps every sound v3 decision and adds the production wrap: measurable success criteria, risk management, determinism guarantees, an error/logging/telemetry contract, device tiering, deep internationalization, SemVer and API stability, a threat model, privacy and age-rating compliance, CI/CD with a release train, update rollback, reproducible builds, traceability, and per-phase estimates.

> ✔ **Bottom line**
>
> Build the core in neutral TypeScript, keep both renderers strictly behind one port, pay for the second engine only when a scene earns it, and wrap the whole thing in the governance and quality systems defined here.

# 3. Audit — What Changed from v3 to v4

v3 scored about **8.5/10 as an architecture** but only **~6.8/10 as a professional program**. The technical design was excellent and is preserved; the gaps were in the production, governance, and compliance layers. v4 targets a defensible 10/10 across every dimension.

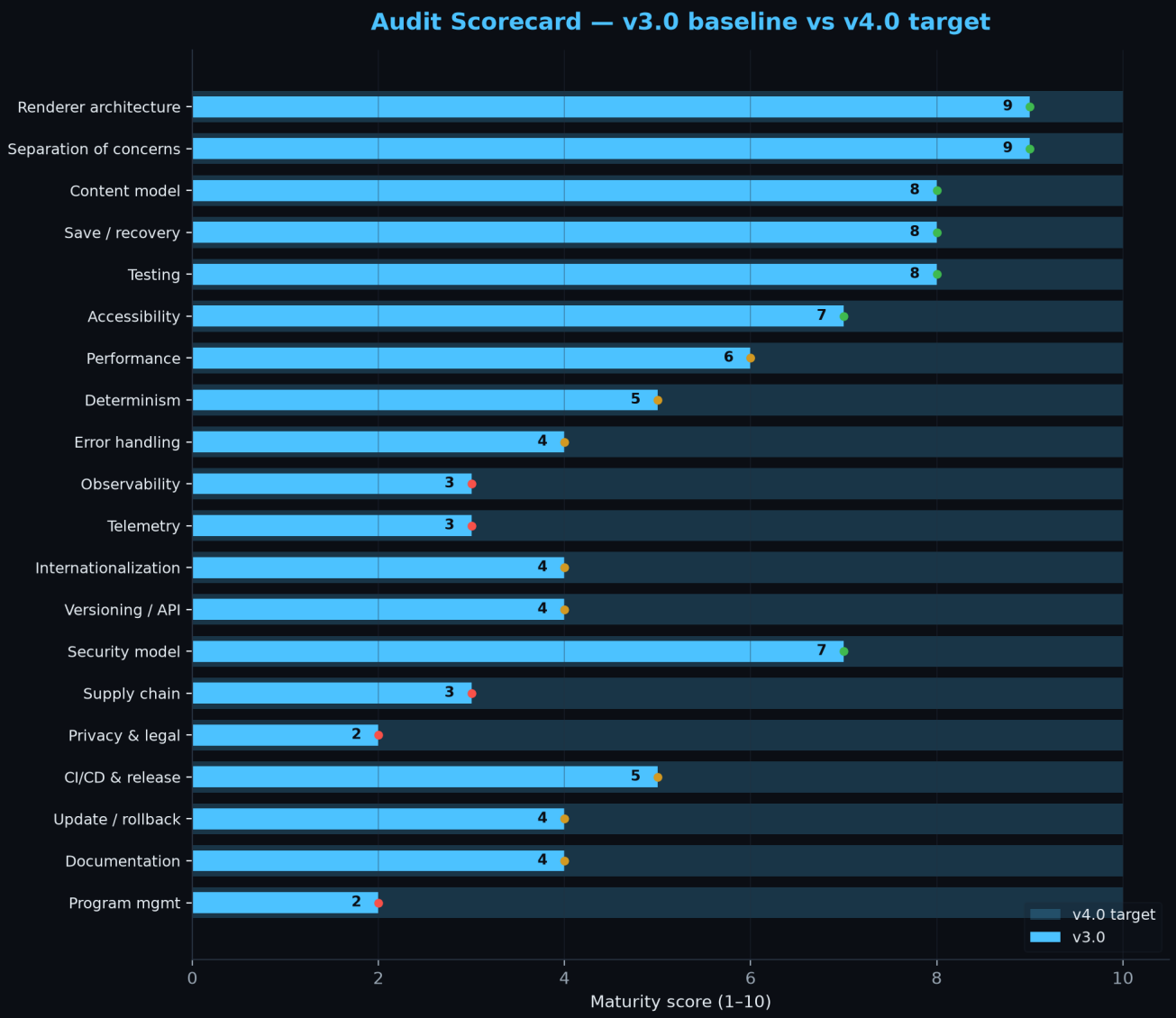



*Figure 3.1 — Maturity scorecard. Solid bars are v3.0; translucent bars are the v4.0 target. Red dots flag the weakest v3 areas.*

## 3.1 Top gaps closed in v4

| Gap in v3 | How v4 closes it |
|---|---|
| No measurable success criteria | Numeric KPIs + budgets, all automatically measured (§2/§5) |
| No risk register / owners | 15-item scored register with owners + triggers (§11) |
| Determinism assumed, not specified | Fixed-step sim, seeded RNG, state hashing, replay harness (§5) |
| ‘Stable error codes’ undefined | Typed Result contract + AREA-NNNN catalog (§5 + appendix) |
| No structured logging | 9-vector logging protocol + schema (§5) |

| Gap in v3 | How v4 closes it |
|---|---|
| Telemetry unspecified | Consent-tiered event catalog, off by default (§5) |
| No performance targets / scaling | Device tiering + dynamic quality + CI budgets (§5) |
| Shallow i18n | ICU MessageFormat, RTL, fonts, text expansion (§5) |
| No API stability policy | SemVer + deprecation + compatibility matrix (§5) |
| No threat model / supply chain | STRIDE + SBOM/SLSA + signing (§6) |
| Zero privacy/legal treatment | GDPR/CCPA, USK/PEGI/ESRB, EULA, OSS attribution (§7) |
| No CI/CD or release train | Full pipeline, staged rollout, tested rollback (§8) |

## 3.2 What v4 deliberately keeps

- The dual-renderer-behind-one-port model and the five coexistence modes.
- The neutral content + command + condition data model.
- Two-phase scene/provider transitions with rollback.
- The save envelope, recovery revisions, and migration discipline.
- The Tauri/Rust least-privilege native boundary.

# 4. Engine & Runtime Architecture

The architecture is a dual-renderer design with one platform-neutral game core, one canonical content model, one RendererPort contract, a Babylon provider, an optional Three provider, and a RendererOrchestrator that selects and owns exactly one provider per live viewport.

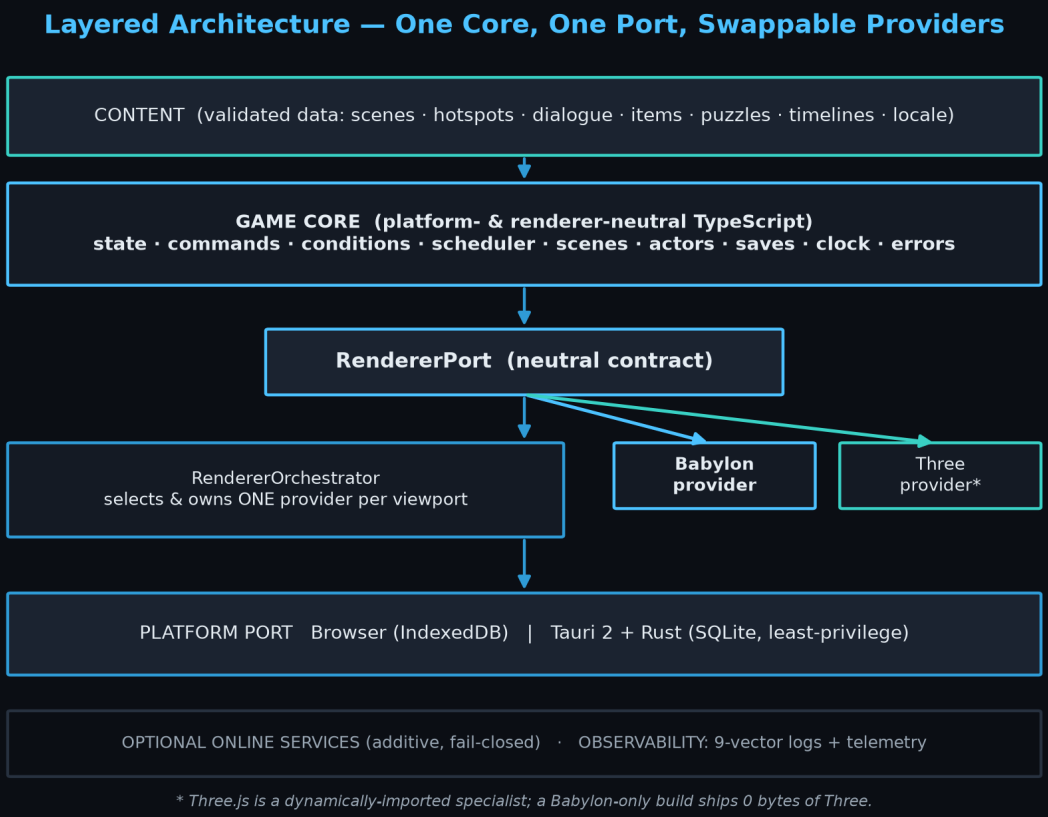



*Figure 4.1 — Layered architecture. Data flows down; the core is renderer- and platform-agnostic.*

## 4.1 Non-negotiable rules (selected)

v4 raises the rule set to **36 lint- or test-enforced invariants** (up from 30). A change to any rule requires an Architecture Decision Record.

| # | Rule | Enforced by |
|---|---|---|
| 2/3 | Core never imports an engine or Tauri | import-boundary lint |
| 6 | Commands/conditions are serializable data | schema + tests |
| 9 | Local gameplay never needs a server | offline e2e |
| 11 | Save writes are versioned + crash-safe | save-recovery test |
| 22/23 | Exactly one provider owns a viewport | contract tests |
| 28 | Babylon-only build ships 0 bytes of Three | bundle gate |
| 31 | No wall-clock drives simulation (v4) | lint + determinism |
| 32 | Boundary errors are typed coded results (v4) | type + tests |
| 33 | Every state change emits a 9-vector log (v4) | log-coverage test |

| # | Rule | Enforced by |
|---|---|---|
| 35 | Deps pinned, attributed, SBOM-listed (v4) | build gate |

## 4.2 Renderer orchestration

The orchestrator is the *only* component allowed to construct, select, or dispose providers and own viewports and animation loops. Selection is capability-based and deterministic (application default → project policy → scene profile), logged with an explanation, and the chosen provider chunk is loaded lazily. Provider transitions are rollback-safe: any pre-commit failure leaves the source viewport intact.

> ◆ **Parity = logical equivalence, not pixels**
>
> Hotspot positions, actor transforms, animation markers, dialogue/puzzle behavior, scene exits and save state must match across renderers. Material appearance, shadows, tone mapping and post may differ. Golden tests compare state hashes and hit maps, not screenshots.

# 5. Engineering Quality Systems

This part is the heart of the v4 upgrade — the systems that make the engine deterministic, observable, fast, and maintainable across years and contributors.

## 5.1 Determinism & the time model

Save/reload identity, renderer parity, and replayable bug reports all depend on a deterministic simulation. All simulation advances from an injected Clock; wall-clock APIs and Math.random() are banned in simulation code. The sim runs on a fixed logical timestep, decoupled from rendering; randomness is seeded with named sub-streams; a canonical serializer yields a stable state hash used by both the replay harness and the parity tests.

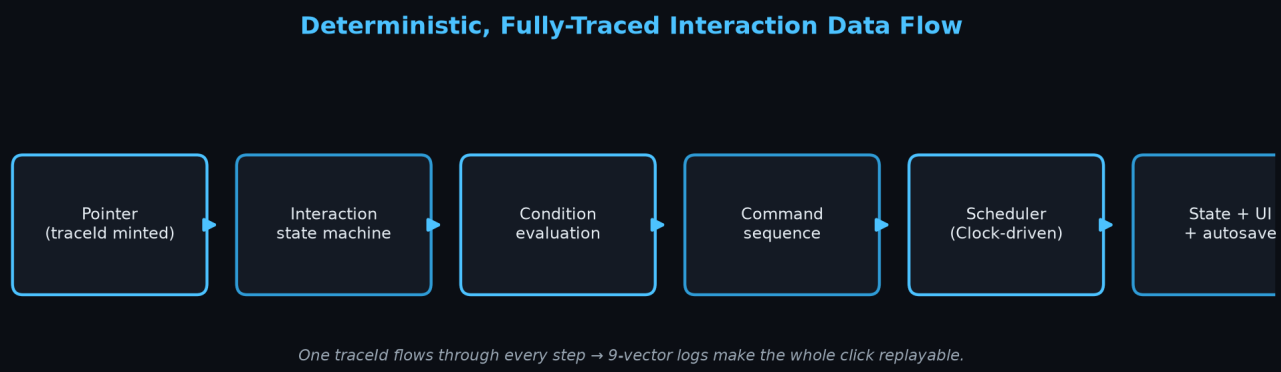



*Figure 5.1 — A single traceId flows through every step, making each interaction replayable and fully logged.*

## 5.2 Typed errors

Boundary-crossing operations return Result<T, AppError> rather than throwing. Each error carries a stable AREA-NNNN code, a severity, a localized user-message key, a retryable flag, and context. Codes are append-only and never reused once published.

## 5.3 Observability — the 9-vector logging protocol

The single biggest v3 gap. Every significant event — every state change and every failure — emits a structured record answering nine forensic questions, correlated by a trace id so one click is reconstructable end to end.


*Figure 5.2 — The nine vectors captured by every log event.*

## 5.4 Telemetry, performance & i18n

- **Telemetry** — off by default; granular, revocable consent tiers; a small typed event catalog; never blocks play.
- **Performance** — numeric budgets (TTI ≤ 3s, 60 FPS simple scenes, autosave p95 ≤ 250ms), device tiering (LOW/MED/HIGH) and dynamic quality scaling with hysteresis. Presentation adapts; simulation never does.
- **Internationalization** — ICU MessageFormat (plurals/gender), full RTL/bidi, font fallback incl. CJK, +40% text-expansion tolerance, pseudo-localization in CI.
- **Versioning** — SemVer for public packages, a deprecation policy, and a per-release compatibility matrix.
- **Memory** — reference-counted bundles with LRU eviction, memory-pressure response, and per-tier GPU budget enforcement.

# 6. Security & Threat Model

*Figure 6.1 — Security, supply-chain integrity, and quality gates wrap the whole product.*

Four trust boundaries: the webview (runs game logic), the Rust native layer (privileged), local storage, and optional remote services. The webview holds no secrets or signing keys; every privileged action crosses into Rust through a narrow, validated, least-privilege command surface.

## 6.1 STRIDE summary

| Threat | Primary mitigation |
|---|---|
| Spoofing | Signed updates, TLS pinning, signed entitlement tokens |
| Tampering | Integrity hashes, signature verification, atomic verified writes |
| Repudiation | 9-vector logs with trace ids for forensics |
| Info disclosure | Redaction allowlist, consent + minimization, sandboxed storage |
| Denial of service | Recovery revisions, safe-mode boot, validation gates |
| Elevation | Least-privilege Tauri capabilities, Rust input validation, no shell |

## 6.2 Supply chain

- Dependencies pinned via committed lockfile; updates land on isolated branches that pass the full suite.
- SBOM (SPDX) generated per release; SLSA-style provenance via reproducible builds.
- Automated CVE scanning — 0 known high/critical at release, or a signed risk-acceptance ADR.
- License scanning + THIRD_PARTY_NOTICES; incompatible licenses fail the build.

# 7. Privacy, Data Protection & Compliance

A shippable professional product must be lawful in its markets (especially EU/DE and US). The product collects **no personal data by default**; nothing leaves the device without explicit, revocable consent.

| Area | v4 posture |
|---|---|
| GDPR / CCPA | Consent basis; access/export/deletion/withdrawal self-service; minimization |
| Consent UX | Granular tiers (none/crash/perf/full), default none, no dark patterns |
| Children | COPPA-style handling, parental gate for external links/purchases |
| Age ratings | USK (DE), PEGI (EU), ESRB (US) descriptor sheet + questionnaires |
| Legal docs | EULA/Terms, Privacy Policy, OSS attribution, accessibility statement |
| Data residency | Prefer EU storage for EU users; TLS in transit, encrypted at rest |

**■ Compliance is a release gate**

Consent works, redaction test passes, policies present and current, ratings metadata complete, licenses cleared — all part of go/no-go.

# 8. Delivery — Build, CI/CD & Release

## 8.1 CI gates (every PR, fail-fast)

- Lint incl. import-boundary + API-extractor; strict typecheck.
- Unit + integration tests; core coverage ≥ 90%.
- Content validation (0 errors); renderer contract + parity tests; determinism replay.
- Bundle gate: Babylon-only build ships 0 bytes of Three.
- Performance budgets; WCAG 2.2 AA automated checks + pseudo-loc screenshots.
- Offline-play e2e; SBOM + reproducible-build check on release branches.

## 8.2 Release train, rollout & rollback

Artifacts are built once, signed, SBOM-attached, and promoted (never rebuilt). Staged rollout rings (internal → canary → broad → full) each clear health gates before the next opens; a regression halts promotion automatically. Every release is revertible, and a boot crash-loop drops into a minimal safe mode.

## 8.3 Testing strategy

| Suite | What it guarantees |
|---|---|
| Unit / integration | Core logic correctness; ≥ 90% coverage on the core |
| Renderer contract | Each provider satisfies the RendererPort behavior |
| Renderer parity | Logical equivalence (state hashes, hit maps, markers) |
| Determinism harness | Record/replay traces produce identical state hashes |
| Performance | Section-2 budgets on reference hardware |
| Accessibility | WCAG 2.2 AA automated + manual audits |
| Memory soak | Bounded growth over long play + many transitions |
| Security | Least-privilege, save-tamper rejection, redaction |

# 9. KPIs & Success Metrics

Every KPI maps to an automated probe — a KPI with no automated measurement is rejected in review.

| Metric | Target | Gate |
|---|---|---|
| Time to interactive (browser, cold) | ≤ 3.0 s | CI perf |
| Sustained frame time (simple scene) | ≤ 16.7 ms | CI perf |
| Three bytes in Babylon-only build | 0 | bundle gate |
| Autosave serialize+write (p95) | ≤ 250 ms | runtime assert |
| Save/reload state divergence | 0 | determinism |
| Core line coverage | ≥ 90% | CI coverage |
| Known high/critical CVEs at release | 0 | CVE scan |
| WCAG 2.2 AA violations | 0 | a11y CI + audit |
| Crash-free session rate | ≥ 99.5% | rollout health |

# 10. Scope, Roles & Risk

## 10.1 Non-goals

- Authoritative multiplayer / netcode.
- Open user-script modding that runs untrusted code at runtime.
- VR/AR head-tracked rendering (possible future specialist provider).
- Built-in storefront / payments / heavy DRM.
- Console platform certification (separate program).

## 10.2 Risk register (highest-severity excerpt)

| ID | Risk | Sev | Mitigation / trigger |
|---|---|---|---|
| R-01 | Dual-engine complexity blows scope | 16 | Babylon-only default; Three needs an ADR |
| R-02 | Three leaks into base bundle | 12 | 0-byte bundle gate; weekly chunk audit |
| R-09 | Determinism breaks (flaky replay) | 12 | Clock+seed; state-hash assertions |
| R-04 | Save corruption / data loss | 10 | Atomic verified writes; recovery revisions |
| R-07 | Privacy / age-rating non-compliance | 10 | Compliance gate; telemetry off by default |
| R-08 | Supply-chain compromise | 10 | Pinned lockfile, SBOM, signing, CVE scan |
| R-11 | Tauri capability over-grant | 10 | Per-window least privilege; security review |

# 11. Implementation Roadmap

Each phase has an explicit **exit gate**; you do not start the next phase until the gate is green. Estimates are engineer-weeks for a 2–3 engineer team at ±30%. Phases 5–9 can partly overlap once Phase 4 is stable.

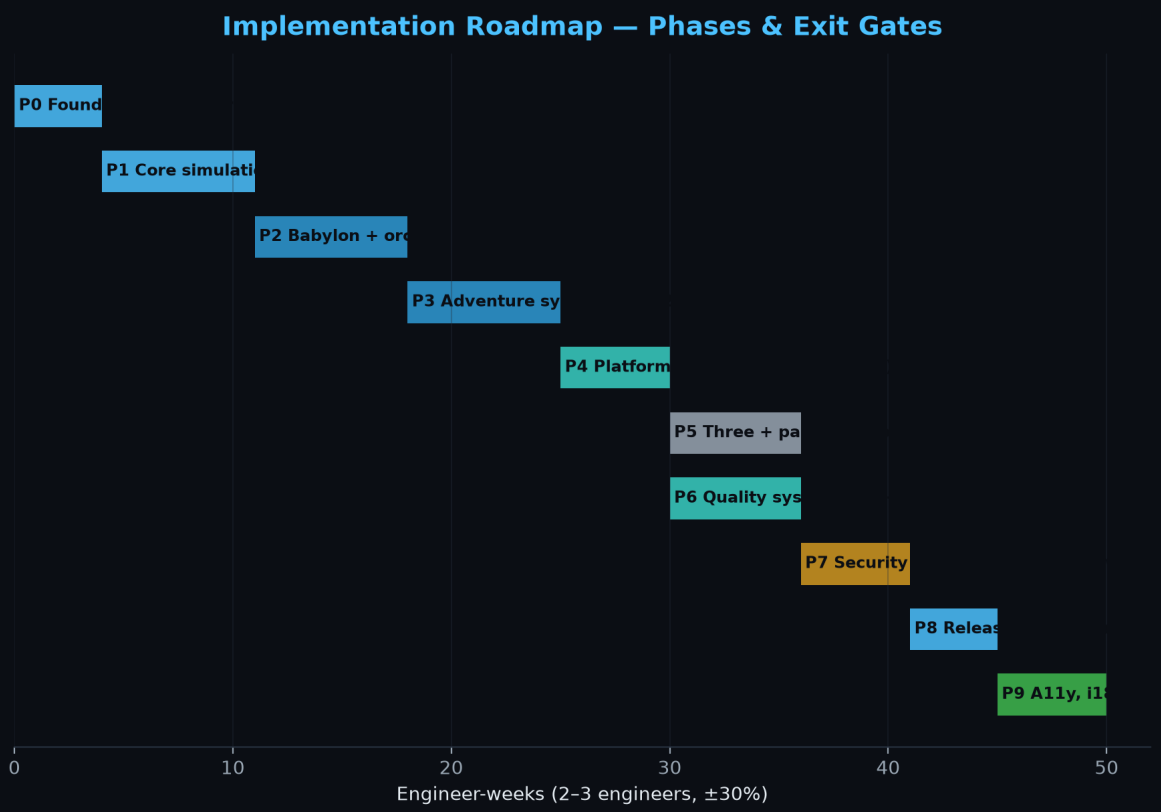



*Figure 11.1 — Phase roadmap. The Three.js phase is optional and only runs if a scene earns it.*

| Phase | Exit gate |
|---|---|
| P0 Foundations | CI runs; boundary rules enforced; hello-scene boots with logging |
| P1 Core sim | Determinism replay = 0 divergence; save identity = 0; coverage ≥ 90% |
| P2 Babylon | Contract tests pass; acceptance scene playable; feedback ≤ 100 ms |
| P3 Adventure | Full demo loop; dialogue/puzzle save+replay; a11y auto-checks pass |
| P4 Platform | Desktop+browser parity; offline e2e; least-privilege review signed |
| P5 Three | Parity 100%; 0-byte bundle gate green |
| P6 Quality | All perf/memory KPIs met; telemetry off-by-default verified |
| P7 Security | 0 high/critical CVEs; compliance checklist; signed reproducible builds |
| P8 Release | Dry-run release canary→full with tested rollback |
| P9 A11y/i18n | 0 AA violations; RTL + a non-Latin language shipped |

# 12. Appendices

## A. Error code catalog (seed)

| Code | Sev | Meaning / default recovery |
|---|---|---|
| SAVE-0001 | CRITICAL | Read-back hash mismatch — keep prior revision, prompt |
| CONT-0007 | CRITICAL | Unresolved content reference — integrity/tamper |
| RNDR-0003 | WARNING | Provider activation failed — fallback engaged |
| CAP-0002 | CRITICAL | Required capability missing, no fallback declared |
| PLAT-0005 | CRITICAL | Privileged command input validation failed |
| SEC-0002 | CRITICAL | Update signature verification failed — abort |
| UPDT-0004 | ERROR | Update apply failed — roll back to current |

## B. Quality preset matrix

| Setting | LOW | MEDIUM | HIGH |
|---|---|---|---|
| Render scale | 0.75x | 1.0x | 1.0x (DPR-capped) |
| Target FPS | 30 | 60 | 60 |
| Shadows | off | basic | cascaded |
| Post-processing | off | minimal | full |
| Texture bundle | low-res | standard | high-res |
| Max loaded scenes | 2 | 3 | 4 |

End of release paper — Version 4.0. The complete normative blueprint (2,130 lines, 61 numbered sections + appendices) ships alongside this paper as a plain-text document.